Bài 1.6 – Làm sạch dữ liệu: dữ liệu thiếu, outlier, encoding

> **Giai đoạn 1 – Python cho ML & Data Science** · Thời lượng: 2 buổi · Hình thức mở đầu: *Tìm lỗi (trong dữ liệu)*

🎯 Sau bài này bạn sẽ

- Phát hiện các loại "bẩn" thường gặp: trùng lặp, sai chính tả, sai kiểu, giá trị vô lý, ô trống, outlier.
- Biết khi nào **xóa**, khi nào **điền**, và điền bằng gì.
- Tìm outlier bằng phương pháp **IQR** và hiểu vì sao z-score có thể "bỏ sót".
- Biến cột chữ thành số (**encoding**) để mô hình ML hiểu được.

Chia buổi:

- **Buổi 1:** Mục 1 → 5 (khám, trùng lặp, chuẩn hóa chữ, đổi kiểu, dữ liệu thiếu).
- **Buổi 2:** Mục 6 → 10 (outlier, encoding, quy trình tổng, bài tập).

> **Sự thật nghề nghiệp:** người làm ML dành khoảng **60–80% thời gian** để làm sạch và chuẩn bị dữ liệu. Mô hình tốt nhất thế giới cũng vô dụng với dữ liệu rác. Người ta gọi đó là **"Garbage In, Garbage Out"** (rác vào, rác ra).

---

1. Thử thách "Tìm lỗi" 🔍 (10 phút)

Bạn nhận được file dữ liệu rao bán căn hộ TP.HCM do 3 nhân viên nhập tay. Hãy tìm **ít nhất 8 vấn đề** trước khi đọc tiếp.

	quan	dien_tich	phong_ngu	huong	noi_that	gia
0	Quận 1	50.0	2.0	Đông	Đầy đủ	6.5 tỷ
1	Bình Thạnh	65.0	2.0	Nam	Cơ bản	4,2 tỷ
2	quận 7	80.0	3.0	Nam	Không	5.1 tỷ
3	Thủ Đức	NaN	3.0	Tây	Cơ bản	3.8 tỷ
4	Q.1	45.0	1.0	None	Đầy đủ	7 tỷ
5	Gò Vấp	60.0	-2.0	Bắc	Không	2.9 tỷ
6	Bình Thạnh	65.0	2.0	Nam	Cơ bản	4,2 tỷ
7	Quận 7	5000.0	3.0	Đông	Đầy đủ	5.6 tỷ
8	Gò Vấp	55.0	2.0	Tây	Cơ bản	3100 triệu
9	Thủ Đức	90.0	NaN	Nam	Không	4.4 tỷ
10	Bình Thạnh	70.0	2.0	Đông	Đầy đủ	4.9 tỷ
11	Gò Vấp	58.0	2.0	N/A	Cơ bản	liên hệ

Tạo dữ liệu này trong Colab:

```
import pandas as pd
import numpy as np
```

```
raw = pd.DataFrame({
    "quan": ["Quận 1", "Bình Thạnh", " quận 7", "Thủ Đức", "Q.1", "Gò Vấp",
             "Bình Thạnh", "Quận 7", "Gò Vấp", "Thủ Đức", "Bình Thạnh", "Gò Vấp"],
    "dien_tich": [50, 65, 80, np.nan, 45, 60, 65, 5000, 55, 90, 70, 58],   #
np.nan = ô trống
    "phong_ngu": [2, 2, 3, 3, 1, -2, 2, 3, 2, np.nan, 2, 2],
    "huong": ["Đông", "Nam", "Nam", "Tây", None, "Bắc", "Nam", "Đông", "Tây",
"Nam", "Đông", "N/A"],
    "noi_that": ["Đầy đủ", "Cơ bản", "Không", "Cơ bản", "Đầy đủ", "Không",
                 "Cơ bản", "Đầy đủ", "Cơ bản", "Không", "Đầy đủ", "Cơ bản"],
    "gia": ["6.5 tỷ", "4,2 tỷ", "5.1 tỷ", "3.8 tỷ", "7 tỷ", "2.9 tỷ",
            "4,2 tỷ", "5.6 tỷ", "3100 triệu", "4.4 tỷ", "4.9 tỷ", "liên hệ"],
})
df = raw.copy()     # luôn làm trên bản sao, giữ nguyên dữ liệu gốc để đối chiếu
```

Đáp án: 10 vấn đề

> Mẹo Notion: chọn tiêu đề "Đáp án: 10 vấn đề" → `Turn into → Toggle heading 3`.

#	Vấn đề	Dòng	Loại lỗi
1	Dòng 6 giống hệt dòng 1	1, 6	Trùng lặp
2	" quận 7" có dấu cách thừa, chữ thường	2	Không nhất quán
3	"Q.1" và "Quận 1" là một	4	Không nhất quán
4	Diện tích trống	3	Dữ liệu thiếu
5	Hướng trống (`None`) và "N/A"	4, 11	Dữ liệu thiếu (2 kiểu viết khác nhau!)
6	Số phòng ngủ = -2	5	Giá trị vô lý
7	Phòng ngủ trống	9	Dữ liệu thiếu
8	Căn hộ 5.000 m²	7	Outlier (chắc là gõ thừa số 0)
9	Giá là chữ, lẫn "4,2" với "4.2", lẫn "tỷ" với "triệu"	nhiều	Sai kiểu dữ liệu
10	Giá "liên hệ"	11	Dữ liệu thiếu (trá hình)

Bạn tìm được mấy lỗi? Nếu được 8 trở lên, bạn đã có "mắt" của người làm dữ liệu.

---

2. Bước 1 – Khám tổng quát

```
print(df.dtypes)              # kiểu dữ liệu từng cột
print(df.isna().sum())        # đếm ô trống mỗi cột
print(df.duplicated().sum())  # đếm số dòng trùng lặp hoàn toàn
print(df["quan"].unique())    # xem mọi giá trị khác nhau của một cột chữ
```

Kết quả mẫu:

```
quan          object
dien_tich    float64
phong_ngu    float64
huong         object
noi_that      object
gia           object
dtype: object
quan         0
dien_tich    1
phong_ngu    1
huong        1
noi_that     0
gia          0
dtype: int64
1
['Quận 1' 'Bình Thạnh' ' quận 7' 'Thủ Đức' 'Q.1' 'Gò Vấp' 'Quận 7']
```

Đọc:

- `gia` có kiểu `object` (chữ), không phải số → phải chuyển.
- `isna()` chỉ thấy **1** ô trống ở `huong`, nhưng thật ra có **2** (thêm "N/A"). Và cột `gia` báo 0 ô trống, dù có "liên hệ". **Dữ liệu thiếu trá hình** không tự lộ ra!
- `unique()` để lộ 7 cách viết cho 5 quận.

---

3. Bước 2 – Xóa trùng lặp

```
df = df.drop_duplicates().reset_index(drop=True)   # xóa dòng trùng, đánh lại số thứ tự 0, 1, 2...
print(df.shape)
```

Kết quả mẫu:

```
(11, 6)
```

Vì sao trùng lặp nguy hiểm? Mô hình "thấy" căn đó 2 lần nên coi nó quan trọng gấp đôi. Tệ hơn, nếu một bản nằm ở tập huấn luyện, bản kia ở tập kiểm tra, mô hình được "xem trước đề thi" (sẽ học kỹ ở bài 1.11).

---

4. Bước 3 – Chuẩn hóa chữ và đổi kiểu dữ liệu

4.1. Chuẩn hóa chữ

```
df["quan"] = (df["quan"]
              .str.strip()                          # bỏ dấu cách thừa ở đầu và
cuối
              .str.title()                          # viết hoa chữ cái đầu mỗi từ:
"quận 7" → "Quận 7"
              .replace({"Q.1": "Quận 1"}))          # thay cách viết tắt bằng tên
chuẩn
print(df["quan"].value_counts())

df["huong"] = df["huong"].replace("N/A", np.nan)  # biến "N/A" thành ô trống thật
sự
```

Kết quả mẫu:

```
quan
Gò Vấp        3
Quận 1        2
Bình Thạnh    2
Quận 7        2
Thủ Đức       2
Name: count, dtype: int64
```

`.str` cho phép dùng các hàm xử lý chuỗi (bài 1.1) trên **cả cột** một lúc.

4.2. Chuyển giá từ chữ sang số

```
def chuyen_gia(chuoi):
    """Đổi '4,2 tỷ' / '3100 triệu' thành số tỷ đồng. Không đọc được thì trả về
NaN."""
    s = str(chuoi).lower().replace(",", ".")       # "4,2 tỷ" → "4.2 tỷ"
    try:
        if "tỷ" in s:
            return float(s.replace("tỷ", "").strip())
        if "triệu" in s:
            return float(s.replace("triệu", "").strip()) / 1000   # triệu → tỷ
    except ValueError:                             # chữ lạ không đổi được thành
số
        pass
    return np.nan                                  # "liên hệ" và mọi trường hợp
khác

df["gia_ty"] = df["gia"].apply(chuyen_gia)
print(df[["gia", "gia_ty"]])
df = df.drop(columns="gia")                        # bỏ cột chữ cũ
```

Kết quả mẫu:

```
           gia  gia_ty
0       6.5 tỷ     6.5
1       4,2 tỷ     4.2
2       5.1 tỷ     5.1
3       3.8 tỷ     3.8
4         7 tỷ     7.0
5       2.9 tỷ     2.9
6       5.6 tỷ     5.6
7   3100 triệu     3.1
8       4.4 tỷ     4.4
9       4.9 tỷ     4.9
10     liên hệ     NaN
```

4.3. Giá trị vô lý → coi như thiếu

```
df.loc[df["phong_ngu"] < 0, "phong_ngu"] = np.nan   # số phòng âm là vô lý → đánh dấu là thiếu
print(df.isna().sum())
```

Kết quả mẫu:

```
quan         0
dien_tich    1
phong_ngu    2
huong        2
noi_that     0
gia_ty       1
dtype: int64
```

Giờ thì mọi ô thiếu (kể cả trá hình) đã lộ diện.

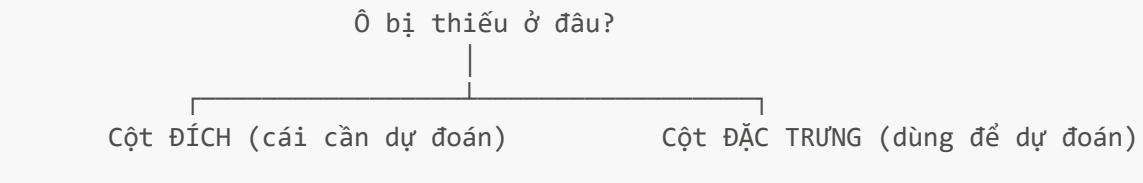
---

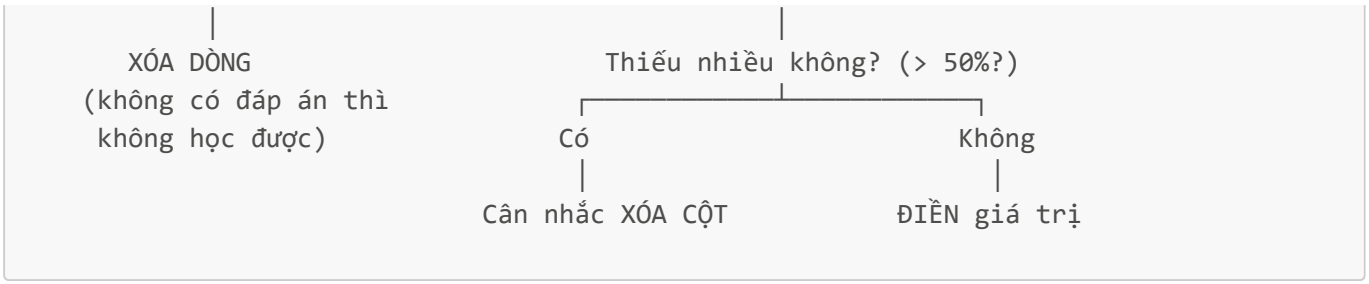
5. Bước 4 – Xử lý dữ liệu thiếu

5.1. Missing Value (Dữ liệu thiếu)

- **Định nghĩa:** Ô không có giá trị, do người nhập bỏ qua, lỗi hệ thống, hoặc thông tin không tồn tại.
- **Ví dụ đời thực:** Form vay tiền, khách bỏ trống ô "thu nhập" vì ngại khai.

5.2. Ba lựa chọn





Kiểu cột	Điền bằng	Vì sao
Số, có outlier hoặc lệch	**Trung vị (median)**	Không bị outlier kéo lệch
Số, phân bố cân đối	Trung bình (mean)	
Chữ / nhóm	**Giá trị phổ biến nhất (mode)** hoặc nhóm mới "Không rõ"	

5.3. Code

```
# (a) Cột đích "gia_ty" thiếu → xóa dòng
df = df.dropna(subset=["gia_ty"]).reset_index(drop=True)
print(df.shape)

# (b) Trước khi điền, tạo cột "cờ" ghi lại dòng nào từng bị thiếu
#     Đôi khi "việc thiếu" cũng mang thông tin (ví dụ: người môi giới cố tình giấu diện tích nhỏ)
df["dien_tich_thieu"] = df["dien_tich"].isna().astype(int)   # True/False → 1/0

# (c) Điền diện tích bằng trung vị. So sánh với trung bình để thấy sự khác biệt
trung_vi = df["dien_tich"].median()
print("Trung vị diện tích:", trung_vi, "| Trung bình:",
round(df["dien_tich"].mean(), 1))
df["dien_tich"] = df["dien_tich"].fillna(trung_vi)            # fillna: điền vào ô trống

# (d) Số phòng ngủ: trung vị
df["phong_ngu"] = df["phong_ngu"].fillna(df["phong_ngu"].median())

# (e) Hướng nhà: tạo nhóm "Không rõ" thay vì đoán bừa
print("Hướng phổ biến nhất:", df["huong"].mode()[0])
df["huong"] = df["huong"].fillna("Không rõ")

print(df.isna().sum().sum())    # tổng số ô trống còn lại trong cả bảng
print(df)
```

Kết quả mẫu:

```
(10, 6)
Trung vị diện tích: 65.0 | Trung bình: 612.8
```

```
Hướng phổ biến nhất: Nam
0
         quan  dien_tich  phong_ngu     huong noi_that  gia_ty  dien_tich_thieu
0      Quận 1       50.0        2.0      Đông   Đầy đủ     6.5                0
1  Bình Thạnh       65.0        2.0       Nam   Cơ bản     4.2                0
2      Quận 7       80.0        3.0       Nam    Không     5.1                0
3     Thủ Đức       65.0        3.0       Tây   Cơ bản     3.8                1
4      Quận 1       45.0        1.0  Không rõ   Đầy đủ     7.0                0
5      Gò Vấp       60.0        2.0       Bắc    Không     2.9                0
6      Quận 7     5000.0        3.0      Đông   Đầy đủ     5.6                0
7      Gò Vấp       55.0        2.0       Tây   Cơ bản     3.1                0
8     Thủ Đức       90.0        2.0       Nam    Không     4.4                0
9  Bình Thạnh       70.0        2.0      Đông   Đầy đủ     4.9                0
```

Nhìn con số này: trung vị **65 m²**, trung bình **612,8 m²**! Chỉ một căn 5.000 m² đã kéo trung bình lên gấp gần 10 lần. Nếu điền bằng trung bình, căn ở Thủ Đức sẽ thành 612,8 m², vô lý. Đó là lý do chọn trung vị.

---

6. Bước 5 – Phát hiện và xử lý outlier

6.1. Outlier (Giá trị ngoại lai)

- **Định nghĩa:** Giá trị khác biệt bất thường so với phần lớn dữ liệu.
- **Ví dụ đời thực:** Trong lớp học có lương phụ huynh trung bình 15 triệu/tháng, có một phụ huynh là tỷ phú.
- **Hai loại:**
  - **Lỗi:** gõ thừa số 0, sai đơn vị → sửa hoặc xóa.
  - **Thật nhưng hiếm:** căn biệt thự thật sự 500 m² → cân nhắc giữ lại.

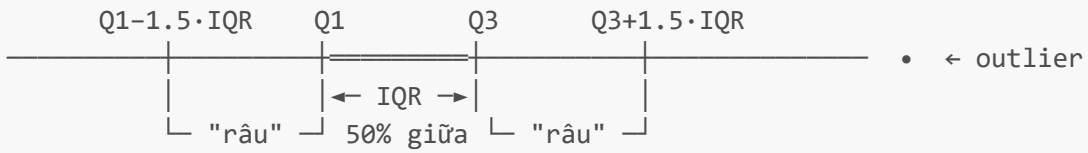
6.2. Phương pháp IQR

Công thức:

```
IQR = Q3 − Q1
Khoảng hợp lệ = [ Q1 − 1.5 × IQR ,  Q3 + 1.5 × IQR ]
```

Ký hiệu	Ý nghĩa
Q1	Phân vị 25%: 25% dữ liệu nhỏ hơn giá trị này
Q3	Phân vị 75%: 75% dữ liệu nhỏ hơn giá trị này
IQR	**Interquartile Range (Khoảng tứ phân vị)**: độ rộng của "50% dữ liệu ở giữa"
1.5	Hệ số quy ước do nhà thống kê John Tukey đề xuất. Chính là "râu" của boxplot (bài 1.5)

Trực giác:



Tính tay với dữ liệu diện tích (10 giá trị sau khi điền): 45, 50, 55, 60, 65, 65, 70, 80, 90, 5000.

- Q1 = 56,25; Q3 = 77,5 (Pandas nội suy giữa các giá trị).
- IQR = 77,5 − 56,25 = **21,25**.
- Cận dưới = 56,25 − 1,5 × 21,25 = 56,25 − 31,875 = **24,375**.
- Cận trên = 77,5 + 31,875 = **109,375**.
- 5000 > 109,375 → **outlier**.

```
Q1 = df["dien_tich"].quantile(0.25)     # phân vị 25%
Q3 = df["dien_tich"].quantile(0.75)     # phân vị 75%
IQR = Q3 - Q1
duoi, tren = Q1 - 1.5 * IQR, Q3 + 1.5 * IQR
print(f"Q1={Q1}, Q3={Q3}, IQR={IQR}, khoảng hợp lệ: [{duoi}, {tren}]")
print(df[(df["dien_tich"] < duoi) | (df["dien_tich"] > tren)])   # in các dòng
outlier
```

Kết quả mẫu:

```
Q1=56.25, Q3=77.5, IQR=21.25, khoảng hợp lệ: [24.375, 109.375]
     quan  dien_tich  phong_ngu huong noi_that  gia_ty  dien_tich_thieu
6  Quận 7     5000.0        3.0  Đông   Đầy đủ     5.6                0
```

6.3. Vì sao không dùng z-score? (Bất ngờ!)

Z-score đo một giá trị cách trung bình bao nhiêu "độ lệch chuẩn". Quy tắc phổ biến: |z| > 3 là outlier.

```
z = (x − trung bình) / độ lệch chuẩn
```

```
z = (df["dien_tich"] - df["dien_tich"].mean()) / df["dien_tich"].std()
print(z.round(2).tolist())
```

Kết quả mẫu:

```
[-0.33, -0.32, -0.31, -0.32, -0.33, -0.32, 2.85, -0.32, -0.3, -0.31]
```

Căn 5.000 m² chỉ có z = **2,85 < 3** → z-score **không bắt được** nó! Vì sao? Chính outlier đã kéo trung bình lên và làm độ lệch chuẩn phình to, nên nó "che giấu" chính mình. Hiện tượng này gọi là **masking (che lấp)**. IQR dùng phân vị nên không bị ảnh hưởng.

Bài học: với dữ liệu ít hoặc lệch, **ưu tiên IQR**.

6.4. Xử lý: xóa, cắt ngọn, hay giữ?

Cách	Khi nào dùng	Code
Xóa	Chắc chắn là lỗi, số lượng ít	`df = df[df["cot"] <= tren]`
Cắt ngọn (clip/capping)	Giá trị thật nhưng quá cực đoan, không muốn mất dòng	`df["cot"].clip(upper=tren)`
Giữ nguyên	Giá trị thật và quan trọng (ví dụ: giao dịch gian lận)	Không làm gì, dùng mô hình chịu được outlier (bài 2.10)

Căn 5.000 m² ở đây rõ ràng là lỗi gõ phím, nên xóa:

```
df = df[df["dien_tich"] <= tren].reset_index(drop=True)
print(df.shape)
```

Kết quả mẫu:

```
(9, 7)
```

> ⚠️ **Không xóa outlier một cách máy móc.** Với dataset California, áp IQR cho cột `mat_do` (người/hộ) sẽ đánh dấu **581 khu** là outlier, vì dữ liệu lệch phải tự nhiên. Xóa hết là mất 3,4% dữ liệu thật. Luôn **nhìn** outlier (bằng boxplot, bài 1.5) và **hỏi vì sao** trước khi xóa.

---

7. Bước 6 – Encoding: biến chữ thành số

Mô hình ML là các công thức toán. Nó không thể nhân "Quận 1" với 0,5. Ta phải đổi chữ thành số.

7.1. Ordinal Encoding (Mã hóa thứ tự)

- **Định nghĩa:** Gán số theo **thứ tự có ý nghĩa**.
- **Dùng khi:** các nhóm có thứ bậc rõ ràng. Ví dụ: nội thất Không < Cơ bản < Đầy đủ; học vấn THPT < Đại học < Thạc sĩ.

```
thu_tu = {"Không": 0, "Cơ bản": 1, "Đầy đủ": 2}     # tự định nghĩa thứ tự
df["noi_that_ma"] = df["noi_that"].map(thu_tu)      # map: thay mỗi giá trị theo
dict
print(df[["noi_that", "noi_that_ma"]].head(4))
```

Kết quả mẫu:

```
  noi_that  noi_that_ma
0   Đầy đủ            2
1   Cơ bản            1
2    Không            0
3   Cơ bản            1
```

7.2. One-Hot Encoding (Mã hóa một-nóng)

- **Định nghĩa:** Mỗi nhóm thành **một cột riêng** chứa 0 hoặc 1. Dòng thuộc nhóm nào thì cột đó bằng 1 ("nóng"), còn lại bằng 0.
- **Dùng khi:** các nhóm **không có thứ tự**. Ví dụ: quận, hướng nhà, màu sắc.

Vì sao không gán Quận 1 = 1, Bình Thạnh = 2, Quận 7 = 3...? Vì mô hình sẽ hiểu "Quận 7 = 3 × Quận 1" hoặc "Bình Thạnh nằm giữa Quận 1 và Quận 7". Hoàn toàn vô nghĩa!

```
  quan                quan_Bình Thạnh  quan_Gò Vấp  quan_Quận 1  quan_Quận 7
quan_Thủ Đức
  Quận 1       ⟶           0             0            1            0
0
  Bình Thạnh   ⟶           1             0            0            0
0
  Quận 7       ⟶           0             0            0            1
0
```

```
# get_dummies: one-hot cho các cột được chỉ định. dtype=int để ra 0/1 thay vì
True/False
df_onehot = pd.get_dummies(df, columns=["quan", "huong"], dtype=int)
print(df_onehot.filter(like="quan_").head(3))    # chỉ xem các cột bắt đầu bằng
"quan_"
print(df_onehot.shape)
```

Kết quả mẫu:

```
   quan_Bình Thạnh  quan_Gò Vấp  quan_Quận 1  quan_Quận 7  quan_Thủ Đức
0                0            0            1            0             0
1                1            0            0            0             0
2                0            0            0            1             0
(9, 16)
```

7.3. So sánh

	Ordinal	One-Hot
Dùng khi	Có thứ tự	Không có thứ tự
Số cột tạo ra	1	Bằng số nhóm
Nhược điểm	Áp sai sẽ tạo thứ tự giả	Nhiều nhóm (ví dụ 700 phường) → bảng phình to
Ví dụ	Nội thất, size áo S/M/L	Quận, hướng, màu

> 💡 Cột có quá nhiều nhóm sẽ có cách xử lý riêng (gom nhóm hiếm, target encoding) ở bài 2.16. Và trong dự án thật, encoding phải "học" từ tập huấn luyện rồi mới áp cho tập kiểm tra (sẽ học kỹ ở bài 1.11 và 2.17).

---

8. Quy trình làm sạch tổng hợp

```
1. Khám       → dtypes, isna().sum(), duplicated(), unique(), describe()
2. Trùng lặp  → drop_duplicates()
3. Chuẩn hóa  → .str.strip(), .str.title(), replace(): gom các cách viết khác nhau
4. Đổi kiểu   → hàm chuyển đổi + apply; dữ liệu thiếu trá hình ("N/A", "liên hệ")
→ NaN
5. Vô lý      → giá trị âm, tuổi 230... → NaN
6. Thiếu      → cột đích: xóa dòng; đặc trưng: điền (median / mode / "Không rõ") +
cột cờ
7. Outlier    → IQR + nhìn biểu đồ → xóa / cắt ngọn / giữ
8. Encoding   → ordinal (có thứ tự) / one-hot (không thứ tự)
```

Dán quy trình này vào Notion làm checklist cho mọi dự án sau.

---

9. Bài tập

Một công ty tài chính có dữ liệu khách vay:

```
kh = pd.DataFrame({
    "tuoi": [25, 34, None, 45, 29, 34, 230],
    "gioi_tinh": ["Nam", "nữ", "Nữ ", "nam", "Nam", "nữ", "Nữ"],
    "thu_nhap": ["15tr", "22 triệu", "18tr", None, "30tr", "22 triệu", "12tr"],
    "nghe_nghiep": ["Văn phòng", "Kinh doanh", "Văn phòng", "Tự do",
                    "Kinh doanh", "Kinh doanh", "Tự do"],
    "tra_no_dung_han": [1, 1, 0, 1, 1, 1, 0],    # cột đích: 1 = trả đúng hạn
})
```

Làm sạch theo quy trình ở mục 8 để ra bảng toàn số:

1. Xóa dòng trùng.
2. Chuẩn hóa `gioi_tinh` thành "Nam"/"Nữ". (Gợi ý: `.str.strip().str.capitalize()`.)
3. Đổi `thu_nhap` thành số triệu đồng, cột mới `thu_nhap_tr`.

4. Tuổi ngoài khoảng 18–100 là vô lý → NaN.
5. Điền thiếu `tuoi` và `thu_nhap_tr` bằng trung vị.
6. Tạo `gioi_tinh_nu` (Nữ = 1, Nam = 0). One-hot cho `nghe_nghiep`. Bỏ các cột chữ.

Câu hỏi thêm: Vì sao `gioi_tinh` chỉ cần **1** cột 0/1 mà không cần 2 cột one-hot?

Đáp án

> Mẹo Notion: chọn tiêu đề "Đáp án" → `Turn into → Toggle heading 3`.

```
kh = kh.drop_duplicates().reset_index(drop=True)                    # 1. dòng 5
trùng dòng 1
kh["gioi_tinh"] = kh["gioi_tinh"].str.strip().str.capitalize()       # 2.
kh["thu_nhap_tr"] = (kh["thu_nhap"]                                  # 3.
                     .str.replace("triệu", "")
                     .str.replace("tr", "")
                     .str.strip()
                     .astype(float))                                 # None vẫn
giữ là NaN
kh.loc[(kh["tuoi"] < 18) | (kh["tuoi"] > 100), "tuoi"] = np.nan      # 4.
print(kh["tuoi"].median(), kh["thu_nhap_tr"].median())               # 31.5 18.0
kh["tuoi"] = kh["tuoi"].fillna(kh["tuoi"].median())                  # 5.
kh["thu_nhap_tr"] = kh["thu_nhap_tr"].fillna(kh["thu_nhap_tr"].median())
kh["gioi_tinh_nu"] = (kh["gioi_tinh"] == "Nữ").astype(int)           # 6.
kh = kh.drop(columns=["thu_nhap", "gioi_tinh"])
kh = pd.get_dummies(kh, columns=["nghe_nghiep"], dtype=int)
print(kh)
```

Kết quả:

```
   tuoi  tra_no_dung_han  thu_nhap_tr  gioi_tinh_nu  nghe_nghiep_Kinh doanh
nghe_nghiep_Tự do  nghe_nghiep_Văn phòng
0  25.0                1         15.0             0                       0
0                     1
1  34.0                1         22.0             1                       1
0                     0
2  31.5                0         18.0             1                       0
0                     1
3  45.0                1         18.0             0                       0
1                     0
4  29.0                1         30.0             0                       1
0                     0
5  31.5                0         12.0             1                       0
1                     0
```

Câu hỏi thêm: Chỉ có 2 nhóm, nên biết "có phải Nữ không" là đủ suy ra "có phải Nam không". Cột thứ hai chỉ lặp lại thông tin. (Tổng quát: k nhóm chỉ cần k−1 cột, sẽ gặp lại tham số `drop_first=True` ở bài 2.2.)

> ⚠️ Lưu ý đạo đức: trong chấm điểm tín dụng thật, dùng giới tính để quyết định cho vay có thể là **phân biệt đối xử** và bị pháp luật nhiều nước cấm. Ở đây chỉ để luyện kỹ thuật.

---

10. Flashcard

Mặt trước	Mặt sau
"Garbage In, Garbage Out" nghĩa là?	Dữ liệu rác thì mô hình cũng ra kết quả rác.
Dữ liệu thiếu trá hình là gì?	"N/A", "liên hệ", -1, 0... trông như có giá trị nhưng thật ra là thiếu.
Cột đích bị thiếu thì làm gì?	Xóa dòng.
Vì sao điền bằng trung vị thay vì trung bình?	Trung vị không bị outlier kéo lệch.
Công thức khoảng hợp lệ IQR?	[Q1 − 1.5·IQR, Q3 + 1.5·IQR], với IQR = Q3 − Q1.
Masking là gì?	Outlier làm phình độ lệch chuẩn, nên z-score không phát hiện được nó.
Ba cách xử lý outlier?	Xóa, cắt ngọn (clip), giữ nguyên.
Ordinal vs One-hot?	Ordinal cho nhóm có thứ tự. One-hot cho nhóm không thứ tự.
Vì sao không đánh số 1, 2, 3 cho các quận?	Mô hình sẽ hiểu sai là có thứ tự, có khoảng cách.

Bài tiếp theo: 1.7 – Thống kê mô tả: trung bình, trung vị, phương sai, phân phối, tương quan.